I need to learn "AI Engineering" book by Chip Hugen fast. Build me a 20-hour plan focused on the 20% that drives 80% of results. Break it into 10 two-hour sessions with the best resources and a 15-minute review at the end of each.

Here's a fast 20-hour plan that focuses on the highest-leverage parts of *AI Engineering* by Chip Huyen: evaluation, prompt/RAG/agents, fine-tuning tradeoffs, dataset engineering, inference optimization, and system design. It is based on the book's chapter structure and emphasis, plus Chip's own discussion that AI engineering shifts toward product, data, and evaluation rather than starting from model training.[1]

What to prioritize

The 80/20 here is:

- Understand the AI engineering workflow.
- Get very strong at evaluation.
- Learn prompt engineering, RAG, and when to use agents.
- Know when not to fine-tune, and what dataset engineering really means.
- Grasp latency/cost tradeoffs and basic inference optimization.
- Be able to sketch a production architecture with feedback loops.[1]

10-session plan

Session	Focus	Best resources	Deliverable
1	AI engineering overview and mental model	Book Ch. 1 summary; Chip's Pragmatic Engineer interview	One-page map of AI engineering vs ML engineering [1]
2	Foundation models and model selection	Book Ch. 2 summary	Decision checklist: API vs open model vs host yourself [1]
3	Evaluation fundamentals	Book Ch. 3 summary	A rubric for good/bad answers for one use case [1]
4	Evaluating systems in practice	Book Ch. 4 summary	Small eval plan: test set, metrics, human review loop [1]
5	Prompt engineering	Book Ch. 5 summary	Prompt template with examples, constraints, and failure cases [1]

Session	Focus	Best resources	Deliverable
6	RAG basics and retrieval choices	Book Ch. 6 summary; BM25 discussion in interview	Retrieval flow diagram: keyword, hybrid, vector, chunking [1]
7	Agents and tool use	Book Ch. 6 summary; interview section on orchestration	Agent design sketch with tools, guardrails, and fallback paths [1]
8	Fine-tuning and dataset engineering	Book Ch. 7 and 8 summaries	"Do I need fine-tuning?" decision tree + data quality checklist [1]
9	Inference optimization	Book Ch. 9 summary	Latency/cost cheat sheet: batching, caching, quantization, parallelism [1]
10	Architecture and feedback loops	Book Ch. 10 summary	End-to-end system design for one AI product idea [1]

Session structure

Each 2-hour session should look like this:

- 35 min: Read the relevant chapter summary or watch the linked talk section.
- 35 min: Take notes on the "why" and "tradeoffs."
- 25 min: Work through a concrete example.
- 10 min: Build a simple artifact, such as a checklist or diagram.
- 15 min: Review and recall without notes, then write 5 bullet takeaways.

That review block matters because the book is broad, and the retention gain comes from repeatedly reconstructing the decision framework, not memorizing every technique.[1]

Best resource order

Use this order for fastest payoff:

1. Book chapter summaries as the main spine.[1]
2. Chip Huyen's Pragmatic Engineer interview for the "how to think" layer, especially evaluation, RAG, and why fine-tuning is usually not first choice.
3. Optional deep dives only when needed:
    - RAG: retrieval basics, BM25, chunking, hybrid search.
    - Evaluation: AI-as-a-judge, comparative eval, human eval.
    - Inference: batching, caching, quantization, TTFT/TPOT.[1]

Session-by-session detail

1) AI engineering overview

Learn how AI engineering differs from classic ML engineering: start from product, then improve data and model choice as needed. Write down three examples of AI apps and the likely architecture for each. End by explaining the difference in your own words in 60 seconds.[1]

2) Foundation models

Focus on what changes performance: data, architecture, scale, post-training, and sampling. The goal is not to train models, but to know how model choices affect capability, cost, and behavior. Make a simple API-vs-hosted decision matrix.[1]

3) Evaluation methodology

This is one of the biggest leverage areas in the book. Learn exact vs subjective evaluation, human evaluation, AI-as-a-judge, and comparative evaluation. Create a rubric for one task such as summarization or support replies.[1]

4) Evaluate AI systems

Turn the rubric into a real evaluation pipeline. Include a small test set, error categories, regression checks, and a process for comparing models or prompts. The key lesson is that without evaluation, everything else is guesswork.[1]

5) Prompt engineering

Treat prompts as interface design. Learn anatomy of a prompt, in-context learning, instruction clarity, examples, and prompt robustness. Build one strong prompt plus a failure-mode version that shows where it breaks.[1]

6) RAG basics

Focus on retrieval before vectors. Chip emphasizes simple retrieval baselines like BM25 and keyword search before jumping to vector databases. Sketch a RAG pipeline that includes chunking, metadata, and hybrid retrieval.[1]

7) Agents and tool use

Learn where agents help and where they create risk. The practical takeaway is that tools plus planning can solve multi-step tasks, but the failure modes expand quickly. Design an agent with a human fallback and explicit guardrails.[1]

8) Fine-tuning and datasets

Learn the question "Should I fine-tune?" before learning how. Most teams should first exhaust prompting, retrieval, and data improvements. Then learn dataset quality, coverage, synthetic data, and when LoRA or PEFT are worth it.[1]

9) Inference optimization

Focus on the big cost and latency levers: batching, caching, parallelism, quantization, and attention efficiency. Learn the meaning of TTFT and TPOT so you can reason about user experience and spend. Write a one-page cost/latency tuning sheet.[1]

10) Architecture and feedback

Put everything together into a production system: model gateway, retrieval, guardrails, observability, and feedback capture. The most important habit is designing the feedback loop so the product gets better over time. Finish by drawing one architecture for a real use case you care about.[1]

How to use the 15-minute review

At the end of each session:

- Write 3 things you now understand.
- Write 2 tradeoffs you can explain.
- Write 1 decision rule you'd use in practice.
- Recall the chapter's main diagram or workflow from memory.
- Convert one note into a checklist you can reuse later.

Would you like me to turn this into a calendar-ready weekly schedule or a Notion-style checklist?

⁂

1. https://github.com/chiphuyen/aie-book/blob/main/chapter-summaries.md
2. https://books.google.com/books/about/AI_Engineering.html?id=S7M1EQAAQBAJ
3. https://www.worldofbooks.com/en-gb/products/ai-engineering-book-chip-huyen-9781098166304
4. https://uk.bookshop.org/p/books/ai-engineering/7727575
5. https://www.goodreads.com/en/book/show/216848047-ai-engineering
6. https://hippocampus-garden.com/book_review_huyen/
7. https://www.youtube.com/watch?v=98o_L3jlixw
8. https://www.goodreads.com/book/show/216848047-ai-engineering
9. https://studylib.net/doc/28266247/848711698-preview-ai-engineering-by-chip-huyen
10. https://play.google.com/store/audiobooks/details/AI_Engineering_Building_Applications_with_Foundati?id=AQAAAEDKHn9epM&hl=en_AU